

Maszyna na kulki

Madina wynalazła maszynę do gry w kulki, aby zabawiać uczestników IOI. Wewnętrzna struktura maszyny przedstawia się następująco:

- Maszyna składa się z N **wierzchołków**, indeksowanych od 0 do $N - 1$. Wierzchołek $N - 1$ nazywany jest **korzeniem**.
- Dla każdego wierzchołka u , dla którego $0 \leq u < N - 1$, **rodzicem** wierzchołka u jest wierzchołek $P[u]$, gdzie $P[u] > u$, a wierzchołek u jest **dzieckiem** wierzchołka $P[u]$. Korzeń nie ma rodzica. Wierzchołek bez dzieci nazywa się **liściem**.

Nie znasz N ani rodzica $P[u]$ żadnego wierzchołka u . Zamiast tego Madina podaje M , liczbę liści i indeksy liści: $0, 1, \dots, M - 1$. Twoim zadaniem jest określenie wewnętrznej struktury maszyny poprzez jej uruchomienie.

Każdy wierzchołek maszyny może pomieścić co najwyżej jedną **kulkę**, a każda kulka ma **wartość**, która jest nieujemną liczbą całkowitą. Mówimy, że wierzchołek ma wartość x jeśli przechowuje kulkę o wartości x . Wierzchołek jest **zajęty**, jeśli zawiera kulkę. W przeciwnym razie jest **wolny**. Początkowo każdy wierzchołek jest wolny.

Można wykonywać dwa rodzaje operacji:

- **insert**(U, X): próba wstawienia nowej kulki o wartości X w liść U .
 - Jeżeli liść U jest zajęty, operacja zwraca `false` bez wstawiania kulki.
 - Jeżeli liść U jest wolny, kulka zostaje umieszczona w wierzchołku U . Następnie kulka wielokrotnie przemieszcza się do rodzica swojego bieżącego wierzchołka, dopóki ten rodzic jest wolny. Ruch zatrzymuje się, gdy kulka dotrze do korzenia lub wierzchołka, którego rodzic jest zajęty. Operacja zwraca wartość `true`.
- **collect**(): jeśli korzeń jest wolny (co oznacza, że maszyna jest pusta), `collect` zwraca pusty ciąg. W przeciwnym razie opisana poniżej rekurencyjna procedura `traverse` zbiera wartości wszystkich piłek aktualnie znajdujących się w maszynie w ciągu S . Początkowo ciąg S jest pusty i wywoływana jest funkcja `traverse(N - 1)`.

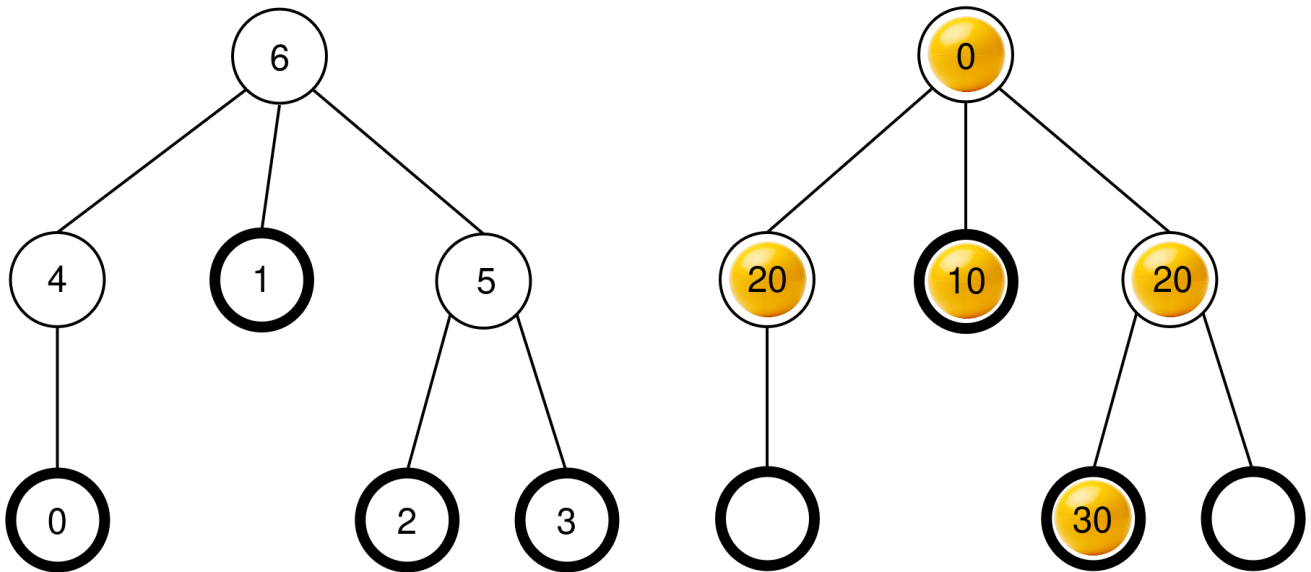
```

traverse(u):
    dodaj wartość kulki w wierzchołku u na koniec ciągu S
    niech c[u] będzie listą zajętych dzieci u
    posortuj c[u] w kolejności niemalejących wartości kulek
        w tych wierzchołkach (jeśli więcej dzieci ma tę samą wartość,
        mogą się pojawić w dowolnej kolejności)
    dla każdego v z c[u]:
        traverse(v)

```

Po zakończeniu tej procedury **wszystkie kulki zostają usunięte** z maszyny, a `collect` zwraca S .

Rozważmy na przykład maszynę z $N = 7$ wierzchołkami i ciągiem rodziców $P = [4, 6, 5, 5, 6, 6]$. Rysunek po lewej stronie przedstawia pustą maszynę z indeksami wierzchołków, natomiast rysunek po prawej stronie przedstawia możliwą konfigurację pitek po pewnej sekwencji operacji `insert` (sekwencja ta jest podana w sekcji Przykłady).



Gdy wywoływana jest funkcja `collect()`, korzeń (wierzchołek 6) jest zajęty, więc S jest inicjowany jako $S = []$ i wywoływany jest `traverse(6)`.

traverse(6): wierzchołek 6 ma wartość 0; dołączenie go do S daje $S = [0]$. Dzieci wierzchołka 6 to 4, 1 i 5, wszystkie one są zajęte. Ich wartości to odpowiednio 20, 10, i 20. Sortowanie w kolejności niemalejącej daje $c[6] = [1, 5, 4]$ (należy zauważyć, że $c[6] = [1, 4, 5]$ również byłoby poprawne, ponieważ wierzchołki 4 i 5 mają tę samą wartość). Procedura wywołuje następnie `traverse` na każdym wierzchołku w $c[6]$ w tej kolejności.

- **traverse(1):** wierzchołek 1 ma wartość 10; dołączenie go do S daje $S = [0, 10]$. Wierzchołek 1 nie ma zajętych dzieci, więc to wywołanie się kończy.

- **traverse(5)**: wierzchołek 5 ma wartość 20; dołączenie go do S daje $S = [0, 10, 20]$. Wierzchołek 5 ma jedno zajęte dziecko (wierzchołek 2), więc $c[5] = [2]$ i procedura wywołuje `traverse(2)`.
 - **traverse(2)**: wierzchołek 2 ma wartość 30; dołączenie go do S daje $S = [0, 10, 20, 30]$. Wierzchołek 2 nie ma zajętych dzieci, więc to wywołanie się kończy.
 - Wykonanie powraca do `traverse(5)` które przetworzyło wszystkie dzieci w $c[5]$, więc jego wywołanie kończy się.
- **traverse(4)**: wierzchołek 4 ma wartość 20; dołączenie go do S daje $S = [0, 10, 20, 30, 20]$. Wierzchołek 4 nie ma zajętych dzieci, więc to wywołanie się kończy.

Wszystkie wywołania zostały zakończone i nie ma więcej wierzchołków do przetworzenia. Na koniec każda kulka zostaje usunięta z maszyny, a `collect` zwraca ciąg $S = [0, 10, 20, 30, 20]$.

Twoim zadaniem jest wyznaczenie liczby wierzchołków N i wygenerowanie ciągu rodziców $R = [R[0], R[1], \dots, R[N-2]]$, która opisuje strukturę maszyny, ale z potencjalnie innym oznaczeniem wierzchołków, które nie są liśćmi ani korzeniem. Formalnie, wygenerowana tablica R jest uważana za poprawną, jeśli każdemu wierzchołkowi u maszyny można przypisać odrębną etykietę $L[u]$ z $0 \leq L[u] < N$ tak aby:

- $L[u] = u$ dla wszystkich $0 \leq u < M$ i $u = N - 1$, i
- $L[P[u]] = R[L[u]]$ dla wszystkich $0 \leq u < N - 1$.

Nie jest wymagane, aby $R[u] > u$. Maszyna **musi być pusta** w momencie raportowania rozwiązania.

Niech K będzie liczbą wykonanych operacji `collect`, a B maksymalną wartością dowolnej kulki spośród wszystkich wywołań `insert`. Niech $C = K + B$. Wówczas C **nie może przekroczyć** 1000. Twój wynik w niektórych podzadaniach zależy od wartości C .

Szczegóły implementacji

Należy zaimplementować następującą procedurę.

```
std::vector<int> find_structure(int M)
```

- M : liczba liści w maszynie.
- Procedura jest wywoływana dokładnie raz dla każdego przypadku testowego.
- Procedura musi zwracać ciąg $R = [R[0], R[1], \dots, R[N-2]]$ o długości $N - 1$, czyli poprawny ciąg rodziców.

Aby nawiązać interakcję z maszyną, Twoja procedura może wywołać następujące dwie procedury.

Pierwsza procedura jest następująca:

```
bool insert(int U, int X)
```

- U : indeks liścia, w którym należy umieścić kulkę. Musi być spełniony warunek $0 \leq U < M$.
- X : wartość kulki. Musi być spełniony warunek $0 \leq X \leq 1000$.
- Ta procedura zwraca `true` jeśli kulka została pomyślnie wstawiona, lub `false` jeśli liść U był już zajęty.
- Tę procedurę można wywołać maksymalnie 500 000 razy.

Druga procedura jest następująca:

```
std::vector<int> collect()
```

- Zbiera wartości wszystkich włożonych kulek, opróżnia maszynę i zwraca utworzony ciąg S .

Jeżeli w dowolnym momencie wykonywania programu wartość C przekroczy 1000, rozwiązanie otrzyma werdykt `Output isn't correct: Too many resources used`.

Struktura maszyny jest ustalona przed wywołaniem funkcji `find_structure`. Program oceniający jest **deterministyczny**: jeśli uruchomisz go dwukrotnie i oba uruchomienia wykonają tę samą sekwencję operacji, wywołania funkcji `collect` zwrócą te same ciągi.

Ograniczenia

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Podzadania

Podzadanie	Punkty	Dodatkowe ograniczenia
1	5	Korzeń ma dokładnie M dzieci.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Brak dodatkowych ograniczeń.

W podzadaniach 3 i 4 Twój wynik zależy od wartości C w następujący sposób.

Podzadanie 3 (25 punktów)

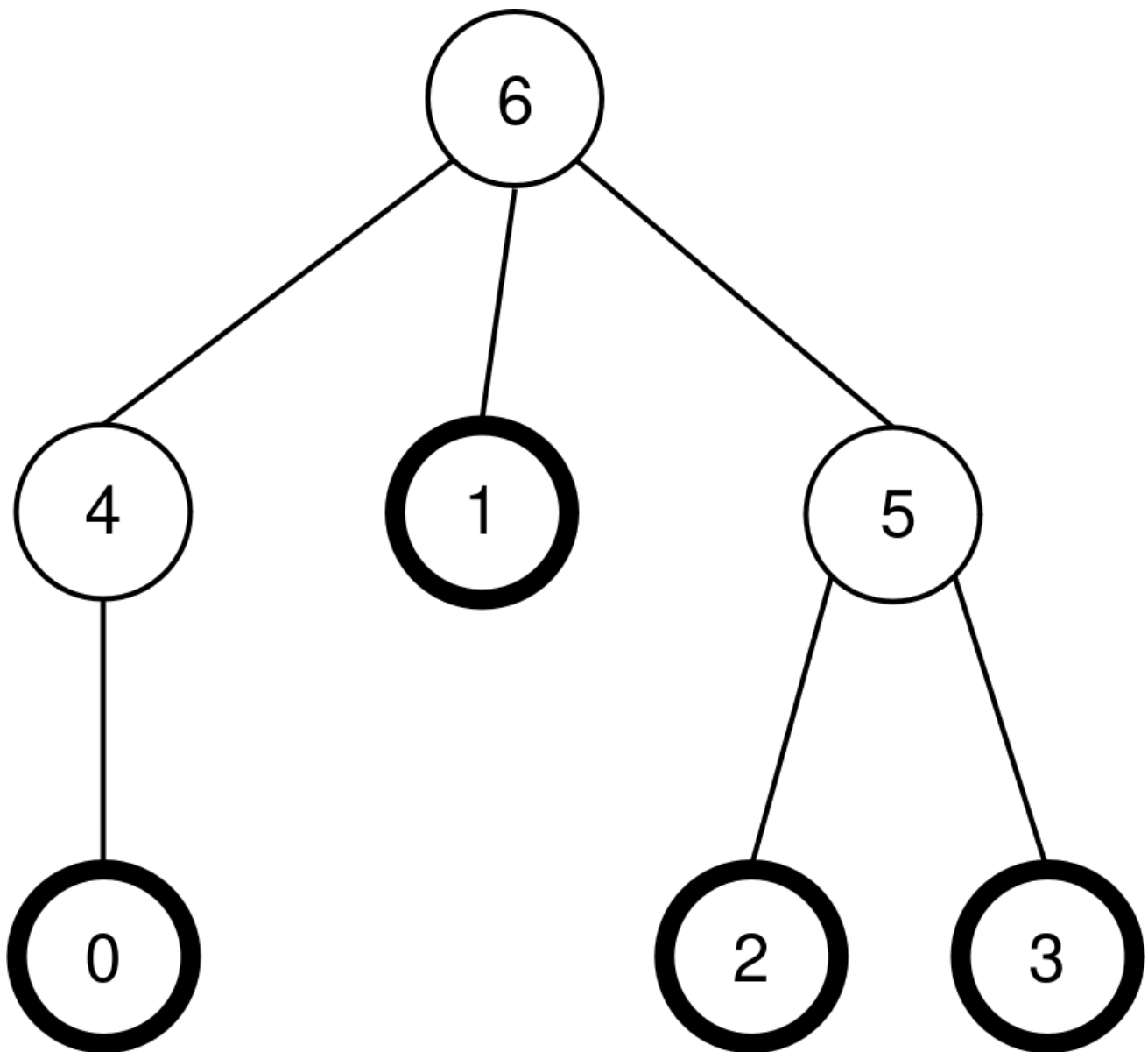
Limity	Punkty
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Podzadanie 4 (60 punktów)

Limity	Punkty
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Przykład

Rozważmy tę samą maszynę, co w opisie zadania, czyli $N = 7$, $M = 4$ i $P = [4, 6, 5, 5, 6, 6]$. Maszynę pokazano ponownie na poniższym rysunku:



Program oceniający wykonuje następujące wywołanie:

```
find_structure(4)
```

Procedura może wykonać następującą sekwencję wywołań:

1. **insert(0, 0)**: liść 0 jest wolny, więc kulka o wartości 0 zostaje umieszczona w liściu 0. Wierzchołek $P[0] = 4$ jest wolny, więc kulka przesuwa się do wierzchołka 4. Wierzchołek $P[4] = 6$ jest wolny, więc kulka przesuwa się do wierzchołka 6. Ponieważ wierzchołek 6 jest korzeniem, ruch zostaje zatrzymany. Wywołanie zwraca `true`.
2. **insert(3, 20)**: liść 3 jest wolny, więc kulka o wartości 20 zostaje umieszczona w liściu 3. Wierzchołek $P[3] = 5$ jest wolny, więc kulka przesuwa się do wierzchołka 5. Ponieważ $P[5] = 6$ jest zajęty, ruch zostaje zatrzymany. Wywołanie zwraca `true`.
3. **insert(1, 10)**: liść 1 jest wolny, więc kulka o wartości 10 zostaje umieszczona w liściu 1. Ponieważ $P[1] = 6$ jest zajęty, kulka pozostaje w wierzchołku 1. Wywołanie zwraca `true`.

4. `insert(2, 30)`: liść 2 jest wolny, więc kulka o wartości 30 zostaje umieszczona w liście 2. Ponieważ $P[2] = 5$ jest zajęty, kulka pozostaje w wierzchołku 2. Wywołanie zwraca `true`.
5. `insert(1, 25)`: liść 1 jest zajęty, więc kulka nie jest umieszczana w liście 1. Wywołanie zwraca `false`.
6. `insert(0, 20)`: liść 0 jest wolny, więc kulka o wartości 20 zostaje umieszczona w liście 0. Wierzchołek $P[0] = 4$ jest wolny, więc kulka przesuwa się do wierzchołka 4. Ponieważ $P[4] = 6$ jest zajęty, ruch zostaje zatrzymany. Wywołanie zwraca `true`.

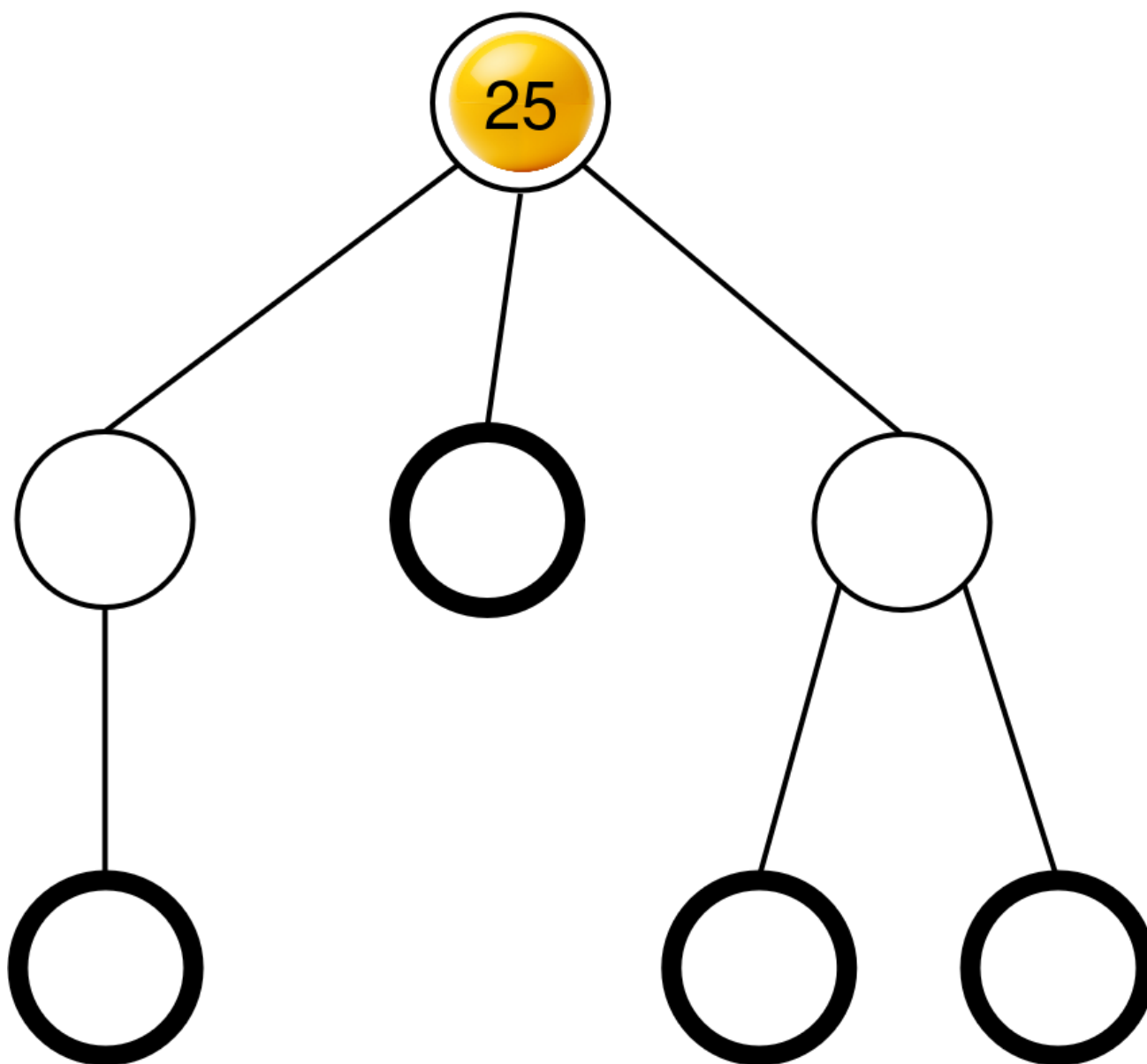
Wynikowa otrzymana konfiguracja kulek była już zilustrowana w opisie zadania.

Następnie procedura może wywołać:

```
collect()
```

Wykonanie tego wywołania zostało już wyjaśnione w opisie zadania, może więc ono zwrócić $S = [0, 10, 20, 30, 20]$. Następnie maszyna jest ponownie pusta.

Następnie procedura może wywołać `insert(2, 25)`. Liść 2 jest wolny, więc umieszczana jest w nim kulka o wartości 25. Kulka ta następnie przemieszcza się do wierzchołka 5, a następnie do wierzchołka 6, gdzie się zatrzymuje. Wywołanie zwraca `true`. Wynikowa konfiguracja kulek jest pokazana na poniższym rysunku.



Na koniec procedura może wywołać `collect()` co zwraca $S = [25]$ i opróżnia maszynę.

Istnieją dwie poprawne ciągi rodziców, które może zwrócić `find_structure`:
 $R = P = [4, 6, 5, 5, 6, 6]$ (co odpowiada etykietowaniu $L = [0, 1, 2, 3, 4, 5, 6]$) i $R = [5, 6, 4, 4, 6, 6]$
 (co odpowiada etykietowaniu $L = [0, 1, 2, 3, 5, 4, 6]$). W tym przykładzie $K = 2$ i $B = 30$, więc
 $C = 32$.

Przykładowy program oceniający

Format wejściowy:

```

N M
P[0] P[1] P[2] ... P[N-2]

```


Format wyjściowy:

```
K B C
Q R[0] R[1] R[2] ... R[Q-1]
```

Wartość Q oznacza długość ciągu R zwróconego przez `find_structure`.